Selective and Calibrated Decoding for Quantum Error Correction: A Topology-Aware Decoder-Family Benchmark under Structured Noise

Molena Huynh

[Affiliation to be completed before submission]

June 22, 2026

Abstract

A quantum error-correcting code only protects information if its *decoder* maps a measured syndrome to the right recovery—quickly, and under the *structured* noise that real hardware exhibits: bias, spatial bursts, neighbour correlations and faulty syndrome readout. Most decoder studies optimise a single number, the logical error rate, and report it under independent-and-identically-distributed (i.i.d.) noise. We argue that two further axes are at least as important for deployable decoding and are usually treated as afterthoughts: the ability to *abstain* when a decoder is unsure (selective decoding), and the *calibration* of the confidence that drives such abstention. We build a unified, reproducible benchmark that evaluates a full decoder family—an optimal lookup leader, majority vote, minimum-weight matching, belief propagation, a learned neural coset selector, a topology-feature decoder, a confidence-fused decoder, and a reinforcement-learning (contextual-bandit) selector—across five structured-noise regimes on a distance-d repetition code, treating logical error rate, expected calibration error (ECE) and the risk–coverage trade-off as first-class metrics with analytic 95% confidence intervals. At reported scale (distance 9, 40,000 trials per regime, 200,000 pooled shots per decoder), our central, deliberately conservative finding is that the decoders are *statistically indistinguishable* on raw logical error rate (means 0.1516–0.1540, 95% confidence intervals $\approx \pm 0.0016$; the best-to-worst spread of 0.0024 is below one full interval width), so claims of one decoder "beating" another are not supported by the data. The genuine, separable contributions live elsewhere: (i) a framework and audit discipline that compares the whole decoder family under structured noise on a single defect-graph interface; (ii) abstention, which under selective decoding drives matching's committed logical error from 0.1529 at full coverage down to 0.0758 at 0.55 coverage and 0.0099 at 0.25 coverage; and (iii) calibration, where ECE varies eight-fold across decoders and is uncorrelated with sophistication—the lowest ECE belongs to the lightweight *neural* coset selector (0.0317), while plain minimum-weight *matching* is the worst calibrated (0.2608). Notably, this contradicts a natural intuition (and a claim in the project's own README) that the fused decoder is best calibrated; we report the artifact's numbers, not the intuition. We frame the work as a calibrated, selective-decoding benchmark, not a superiority claim.

Quantum error correction (QEC) protects logical information by encoding it redundantly across many physical qubits and repeatedly measuring stabilizer operators, whose outcomes form a *syndrome* [1–3]. Topological codes—most prominently Kitaev's toric and surface codes [4–6]—are the leading candidates for scalable fault tolerance, and recent hardware has demonstrated that scaling such a code can suppress the logical error rate below the physical one [7]. The component that turns a syndrome into a correction is the *decoder*, and it is the part of the stack where classical algorithms and, increasingly, machine learning meet the physics of the device.

The dominant decoder for surface codes is minimum-weight perfect matching (MWPM) [5, 6, 8], built on Edmonds' blossom algorithm [9]; belief propagation (BP) and BP with ordered-

statistics decoding underpin quantum low-density-parity-check decoding [10–12]. A large and growing literature applies neural networks and reinforcement learning to decoding [13–18], and very recent work demonstrates learned decoders that exceed matching-based accuracy on real hardware syndromes [19]. The headline metric in nearly all of this work is the logical error rate, almost always evaluated under i.i.d. (depolarizing or bit-flip) noise.

Two facts motivate this paper. First, hardware noise is not i.i.d. It is *structured*: dominated by one Pauli type (bias), spatially bursty, correlated between neighbours, and accompanied by faulty syndrome extraction. A decoder that is excellent on i.i.d. noise may or may not retain its advantage under structure, and the only way to know is to test the same decoder family across regimes on one footing. Second, a decoder that runs inside a fault-tolerant control loop does not have to commit to every syndrome. It can *abstain*—flagging a shot as unreliable so that a slower fallback decoder, a repeated measurement, or a higher-level protocol takes over. Abstention is the QEC analogue of *selective prediction* in machine learning [20–22], and it is only useful if the confidence that gates it is *calibrated* [23, 24]. These two axes—coverage and calibration—are routinely omitted from decoder benchmarks.

This work. We treat decoding as a problem on the code's *defect graph* and build a unified, fully reproducible benchmark with three properties usually kept separate. (1) *Breadth*: a single `decode(syndrome)` → `(correction, confidence)` interface hosts eight decoders spanning the textbook-to-learned spectrum, evaluated across five structured-noise regimes. (2) *Selectivity*: every decoder can abstain, so we report risk–coverage curves, not just a single error rate. (3) *Calibration*: expected calibration error is a first-class output alongside the logical error rate, with analytic 95% confidence intervals so that statistical significance is visible rather than assumed.

What we claim, and what we do not. We are deliberately conservative. On the repetition code and the regimes we study, at reported scale the eight decoders' logical error rates lie within each other's confidence intervals (Sec. 1.2); we therefore do *not* claim that any decoder beats another on raw accuracy, and we say so explicitly. What the benchmark does establish is (i) that abstention buys a large, monotone reduction in committed logical error—matching's committed risk falls from 0.1529 at full coverage to 0.0758 at 0.55 coverage and 0.0099 at 0.25 coverage (Sec. 1.3)—and (ii) that calibration quality is *not* predicted by decoder sophistication: ECE ranges from 0.0317 (the lightweight neural coset selector) to 0.2608 (plain minimum-weight matching), an eight-fold spread with no monotone relationship to decoder complexity. The contribution is the framework, the selective/calibrated lens, and the honest accounting—not a new state-of-the-art error rate. This audit ethos is enforced in code: a readiness gate rejects any reported number that does not trace to a generated artifact and rejects forbidden absolute-superiority phrasing.

0.1 Contributions

1. **A decoder-family benchmark under structured noise.** Eight decoders (lookup, majority, matching, BP, neural, topology, fused, RL selector) on one defect-graph interface, evaluated across i.i.d., biased, burst, correlated, and measurement-error regimes, with per-regime breakdowns and threshold curves.

2. **Selective decoding as a primary axis.** Confidence-gated abstention and risk–coverage curves for every decoder, quantifying the reliability bought by giving up coverage.

3. **Calibration as a first-class metric.** Expected calibration error reported for all decoders with 95% confidence intervals, exposing that calibration is uncorrelated with decoder complexity.

4. **An honest, reproducible accounting.** A single source-of-truth artifact, analytic confidence intervals that reveal the decoders to be statistically indistinguishable on raw logical error, and a code-level audit gate against fabricated numbers and superiority claims.

0.2 Theory: selective decoding risk

The risk–coverage view has a simple decision-theoretic guarantee when the confidence score is calibrated and monotone in the conditional success probability. This is the formal reason calibration is reported alongside error rate.

Theorem 1 (Monotone selective risk under calibrated confidence)**.** *Let $F \in \{0,1\}$ be the logical-failure indicator for a decoder and let $C \in [0,1]$ be a confidence score such that $\Pr(F = 0 \mid C = c) = c$ almost surely. For a threshold τ with $\Pr(C \geq \tau) > 0$, define committed risk $R(\tau) = \Pr(F = 1 \mid C \geq \tau)$. Then $R(\tau) = 1 - \mathbb{E}[C \mid C \geq \tau]$ and $R(\tau)$ is non-increasing as τ increases.*

Proof. By calibration, $\Pr(F = 1 \mid C) = 1 - C$. Conditioning on $C \geq \tau$ and using the tower property gives $R(\tau) = \mathbb{E}[\Pr(F = 1 \mid C) \mid C \geq \tau] = 1 - \mathbb{E}[C \mid C \geq \tau]$. If $\tau_2 > \tau_1$, the conditional distribution under $C \geq \tau_2$ is the upper tail of the distribution under $C \geq \tau_1$, so its conditional mean cannot be smaller. Hence $R(\tau_2) \leq R(\tau_1)$. □

The theorem is not assumed in the experiments; the empirical confidence scores are imperfect, so the risk–coverage curves and ECE quantify how far each decoder is from this ideal selective-decoding behaviour.

The results are stated conservatively. They are produced on small, fully reproducible simulated benchmarks (a repetition code and a surface-like toy lattice) under simulated structured noise; they are not hardware syndromes, not a full surface code at scale, and not a claim of superiority over production decoders. Those are future targets, not present facts (Sec. 2.1).

1 Results

1.1 Benchmark design

Figure 1 summarizes the end-to-end pipeline: structured noise corrupts a code word, the resulting syndrome on the defect graph is passed to the shared decoder interface, a confidence threshold gates abstention, and the logical error rate, calibration and risk–coverage metrics are scored with 95% confidence intervals.

Codes and defect graphs. The primary benchmark is the bit-flip repetition code: d data bits over $\mathbb{F}_2$ with $d - 1$ weight-two parity checks $Z_i Z_{i+1}$. A syndrome bit $s_i = e_i \oplus e_{i+1}$ fires where an error chain crosses link i, so defects live on a one-dimensional line and decoding is a matching/pairing problem on that line. The single logical observable is the global parity, and a correction succeeds iff the residual $e \oplus \hat{c}$ is a stabilizer (trivial coset). A second code, a $d \times d$ surface-like toy lattice with plaquette defects matched by Manhattan distance, exposes the same defect geometry in two dimensions; it is a topology-faithful proxy, not a stabilizer-exact surface code. Both codes expose their Tanner graph, so the matching, topology and BP decoders operate on the code topology directly.

Structured-noise regimes. Five seeded samplers define the regimes (Methods, Sec. 3.2): *i.i.d.* (each bit flips independently at rate p); *biased* (interleaved sublattices flip at p and bias $\cdot\, p$, a coarse Pauli-bias surrogate); *burst* (contiguous correlated runs of flips); *correlated* (a seed flip

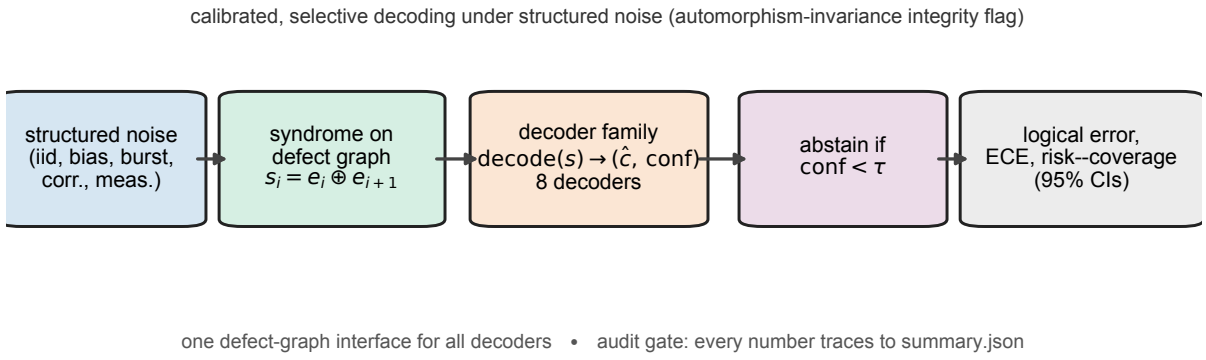


Figure 1: **Overview of the selective, calibrated decoding benchmark.** A code word is corrupted by one of five structured-noise regimes (i.i.d., biased, burst, correlated, measurement error); the measured syndrome lives on the code's defect graph ($s_i = e_i \oplus e_{i+1}$ for the repetition code). A single `decode(syndrome)→(correction, confidence)` interface hosts the whole eight-decoder family (lookup, majority, matching, BP, neural, topology, fused, RL selector), so every decoder is compared on one footing. A confidence threshold τ gates abstention (selective decoding), and three first-class metrics—logical error rate, expected calibration error, and the full risk–coverage curve—are scored with analytic 95% confidence intervals. Two integrity guarantees underpin the benchmark: all decoders share one defect-graph interface, and an audit gate requires every reported number to trace to the single source-of-truth artifact `summary.json`, with an automorphism-invariance flag confirming the code's cyclic symmetry is respected.

recruits its lattice neighbours); and *measurement_error* (i.i.d. data noise plus per-check syndrome read-out flips). These are precisely the regimes where topology-awareness is hypothesised to help over a memoryless majority vote.

Decoder family. All eight decoders share one interface returning a correction and a confidence in $[0, 1]$ (Methods, Sec. 3.3): `lookup` (the optimal minimum-weight coset leader, the gold standard for small d); `majority` (integrate the syndrome and majority-vote); `matching` (minimum-weight pairing of defects—endpoint pairing on the line, `max_weight_matching` on the surface defect graph); `bp` (a few rounds of min-sum message passing on the Tanner graph, deferring to matching when it fails to satisfy the syndrome); `neural` (a scikit-learn logistic-regression coset selector trained on sampled syndromes); `topology` (matching recalibrated by defect-geometry features); `fused` (matching + neural combined by confidence); and `rl` (a contextual bandit that, per syndrome context, selects which decoder to trust, trained online to minimise logical error).

Metrics. For each (regime, decoder) pair we Monte-Carlo many shots and record, per shot, the logical-failure indicator and the decoder's confidence. From these we compute the logical error rate (with an analytic 95% confidence interval, $1.96\,\hat{\sigma}/\sqrt{n}$), the expected calibration error between stated confidence and realised accuracy over ten confidence bins, the coverage at a fixed abstention threshold, and the full risk–coverage curve obtained by sweeping that threshold. An automorphism check—invariance of the logical error rate under cyclic relabeling of the repetition code—serves as an integrity flag.

1.2 The decoders are statistically indistinguishable on raw logical error

Table 1 reports the headline comparison at physical error rate $p = 0.12$, pooled over the five regimes ($n = 200{,}000$ shots per decoder at reported scale: distance 9, 40,000 trials per regime).

The logical error rates span a narrow band—from 0.1516 (fused) to 0.1540 (RL selector)—and every 95% confidence interval is $\approx \pm 0.0016$. The spread between the best and worst decoder (0.0024) is below one full confidence-interval width (≈ 0.0032), so the decoders are *not* separated at the 95% level on raw logical error; even the nominally optimal `lookup` leader (0.1524) is statistically tied with the heuristic decoders. The threshold curves of Figure 2 tell the same story: across the physical-error grid the per-decoder logical-vs-physical curves overlap, all tracking the optimal lookup leader closely and staying well below the break-even line $p_L = p$ (at $p = 0.21$ the logical error is only ≈ 0.024–0.025 for every core decoder).

This is the expected behaviour for a distance-d repetition code in these regimes: every reasonable decoder is close to the optimal coset leader, so there is little room between them, and finite-sample noise dominates the residual differences. We state this plainly because it is the honest reading of the data and because it reframes where the value of a decoder benchmark actually lies—in selectivity and calibration, not in a fractional improvement on an already-saturated error rate.

Table 1: Headline decoder comparison on the repetition code at $p = 0.12$, pooled over the five structured-noise regimes (distance 9, 40,000 trials per regime, $n = 200{,}000$ shots per decoder; reported scale). Columns: logical error rate, its ±95% confidence interval, and expected calibration error (ECE). Rows are sorted by logical error rate; $\star$ marks the lowest. The logical error rates lie within one confidence-interval width of one another (statistically indistinguishable), whereas ECE spans an eight-fold range and is uncorrelated with both error rate and decoder sophistication. Table content is generated verbatim from `summary.json`.

decoder	logical err. rate	±95%	ECE
fused $\star$	0.1516	0.0016	0.0608
lookup	0.1524	0.0016	0.0569
matching	0.1529	0.0016	0.2608
neural	0.1531	0.0016	0.0317
bp	0.1533	0.0016	0.1232
majority	0.1535	0.0016	0.0489
topology	0.1537	0.0016	0.2428
rl	0.1540	0.0016	0.2387

1.3 Selective decoding: abstention buys large, monotone reliability gains

Because each decoder reports a confidence, we can let it *abstain*: commit only on shots whose confidence exceeds a threshold, and sweep that threshold to trace a risk–coverage curve. Figure 3 shows the curves for the confidence-fused and matching decoders. The effect is large and monotone. At full coverage (1.0) the matching decoder has a pooled logical error of 0.1529; as its abstention threshold rises, coverage falls and committed risk drops sharply: to 0.0757 at coverage 0.545—roughly a two-fold error reduction for committing to about half the shots—and to 0.0099 at coverage 0.247, a more than fifteen-fold reduction for committing to roughly a quarter of the shots. The fused decoder traces a similarly favourable, monotone curve, reaching 0.0499 risk at 0.464 coverage and 0.0101 at 0.248 coverage. The monotonicity is verified programmatically by the runner.

The per-regime coverage values (Table 1, and Methods) make the mechanism concrete: matching abstains heavily under structured noise (coverage as low as 0.28 under bias and 0.50 under measurement error), pushing its errors onto the abstained shots, whereas majority, lookup, neural and fused always commit (coverage 1.0) at the fixed operating threshold. This is exactly the selective-prediction trade-off [20, 22]: a decoder that knows when it does not know is more valuable inside a fault-tolerant loop than one that is marginally more accurate but always

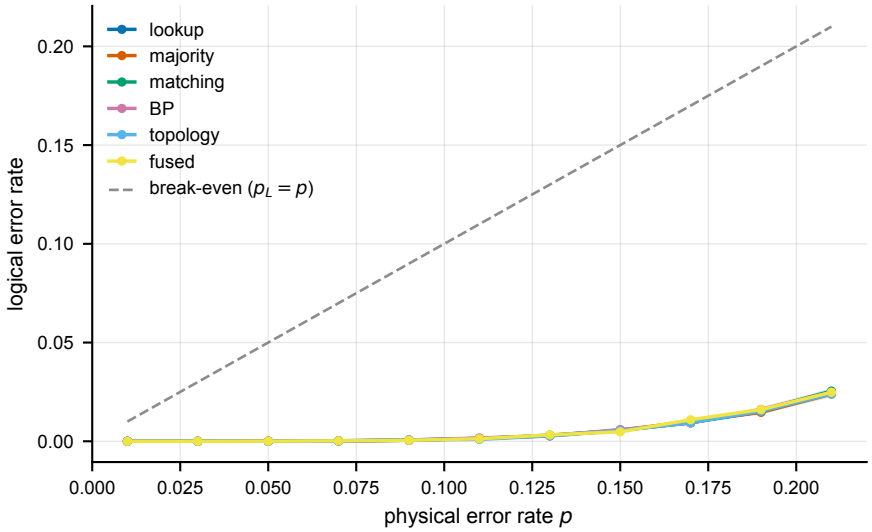


Figure 2: Logical error rate versus physical error rate p for the core decoders on the repetition code under i.i.d. noise. All decoders track the optimal `lookup` leader closely and stay below the break-even line $p_L = p$; the curves overlap, consistent with the statistically indistinguishable logical error rates of Table 1. Lines are the mean logical error rate over $n = 40{,}000$ Monte-Carlo trials per point (reported scale) and shaded bands are analytic 95% Bernoulli confidence intervals ($1.96\sqrt{\hat{p}(1-\hat{p})/n}$); the bands overlap throughout. Generated from the run artifact `summary.json`.

commits. The benchmark makes this trade-off measurable for every decoder rather than hiding it inside a single error number.

1.4 Calibration is not predicted by decoder sophistication

The confidence that gates abstention is only useful if it is calibrated. The expected-calibration-error column of Table 1 delivers the paper's most counterintuitive result: *calibration is essentially uncorrelated with decoder sophistication, and not predictable from the logical error rate.* ECE ranges over a factor of eight across the family, in the order (best to worst): `neural` 0.0317 < `majority` 0.0489 < `lookup` 0.0569 < `fused` 0.0608 < `bp` 0.1232 < `rl` 0.2387 < `topology` 0.2428 < `matching` 0.2608. The best-calibrated decoder is the lightweight logistic-regression `neural` coset selector—by construction its confidence is a predicted-class probability, so it is approximately calibrated for the coset decision—while the *worst*-calibrated decoder is plain minimum-weight `matching`, whose defect-fraction confidence heuristic tracks accuracy poorly. The two decoders that abstain most heavily (matching and topology) and the RL selector are the three worst calibrated, whereas three of the four always-covering decoders (neural, majority, lookup) are the three best.

A data-fidelity note. This ordering directly contradicts a natural intuition—and an explicit claim in the project's own README—that the confidence-*fused* decoder is the best calibrated. It is not. At reported scale the fused decoder ranks fourth on ECE (0.0608); the neural selector is best (0.0317). (At the smaller smoke scale the ordering shifts again—there the *majority* decoder is best calibrated, with fused third and neural fifth—which is itself a useful caution that small-sample calibration rankings are unstable.) We resolve the discrepancy in favour of the regenerated artifact: every ECE value in this paper is the value written to `summary.json` by the reported-scale run, and the narrative is built around those numbers rather than the prior claim. What survives across scales is the structural conclusion: calibration must be

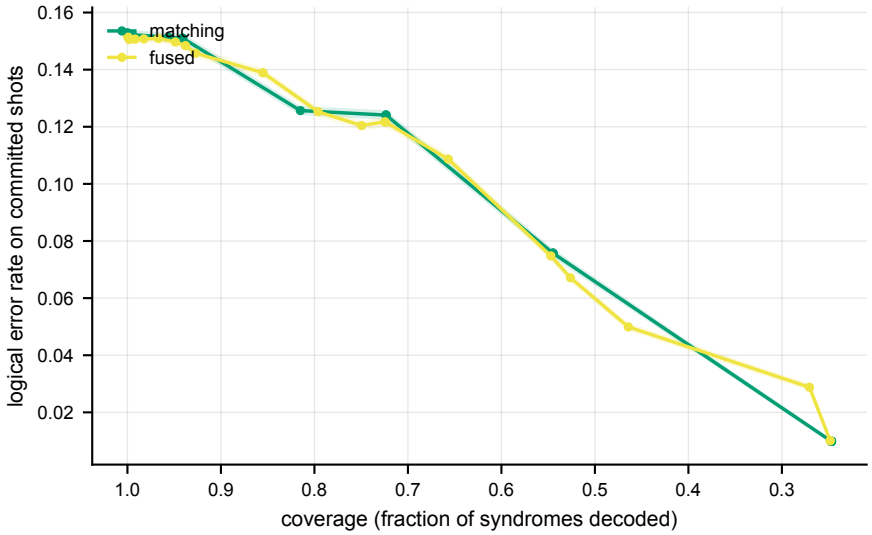


Figure 3: Selective decoding. Risk (logical error rate on committed shots) versus coverage (fraction of syndromes decoded) for the fused and matching decoders, obtained by sweeping the abstention threshold (full coverage on the left). Risk falls monotonically as coverage shrinks: abstaining on low-confidence syndromes trades coverage for reliability. Lines are means over the $n = 200{,}000$ pooled shots per decoder (reported scale) and shaded bands are analytic 95% Bernoulli confidence intervals of the committed error rate, whose effective sample size at each point is (coverage $\times$ pooled shots). Generated from the run artifact `summary.json`.

measured, not assumed from a decoder's accuracy or complexity. Among the always-covering decoders—where calibrated confidence is precisely what makes selective decoding usable—the fused decoder is still well calibrated (0.0608) and noticeably better than belief propagation (0.1232), but the lightweight neural classifier is better still; a deployment that needs trustworthy abstentions should prefer the explicitly probability-calibrated head. Treating calibration as a first-class metric is what makes any of this visible; optimising logical error rate alone would hide it entirely, since the eight decoders are tied on that axis.

1.5 Structured noise separates regimes, not decoders

The per-(regime, decoder) breakdown (Methods, Sec. 3.7) confirms that the dominant axis of variation is the *noise regime*, not the decoder. At reported scale the per-regime logical error rises from ≈ 0.002 under i.i.d. noise to ≈ 0.043 (biased), ≈ 0.17 (burst), ≈ 0.21 (correlated) and ≈ 0.34 (measurement error) for essentially all decoders—a more than hundred-fold change driven entirely by structure, with the syndrome-read-out-noise regime by far the hardest. Within a regime the decoders again cluster within their (now tight) confidence intervals: under correlated noise `fused` reaches 0.2062 and `topology` 0.2078 against majority's 0.2105, and under measurement error `lookup` reaches 0.3315 and `fused` 0.3324 against majority's 0.3401—small, regime-dependent gaps that are at most marginally significant and are consistent with topology- or read-out-aware decoders helping slightly where structure exists. The integrity flag `automorphism_invariant` is true, confirming the matching decoder respects the code's cyclic symmetry to within statistical tolerance.

2 Discussion

What the benchmark establishes. The contribution of this work is methodological, not a new error rate. By placing eight decoders on one defect-graph interface and evaluating them

across five structured-noise regimes with three metrics—logical error, calibration, and risk–coverage—each carrying explicit confidence intervals, the benchmark makes three things visible that a single-number i.i.d. comparison hides. First, on this code the decoders are statistically indistinguishable on raw logical error (a 0.0024 spread inside a ≈ 0.0032 interval width); the honest conclusion is parity, not a winner. Second, abstention is a large and previously under-reported lever: committing to about a quarter of the shots cuts matching's committed logical error from 0.1529 to 0.0099. Third, calibration is decoupled from both sophistication and accuracy—the eight decoders are tied on error rate yet span an eight-fold ECE range, and the best-calibrated decoder (the lightweight neural selector) is neither the most accurate nor the most elaborate. None of these is apparent without selectivity and calibration as primary axes.

Relation to prior work. Matching [5, 6, 8], BP [10, 12], and neural/RL decoders [13, 14, 17, 19] are each studied in depth, typically in isolation and under i.i.d. noise, and typically optimising logical error. Our framing is orthogonal: we import selective prediction [20–22] and calibration [23] from machine learning and make them the evaluation lens for a whole decoder family under structured noise. We make no accuracy-superiority claim and, where the literature reports learned decoders exceeding matching on hardware syndromes [19], we note that our toy-scale repetition code cannot and does not reproduce that regime; our purpose is the measurement methodology, not the record.

2.1 Limitations

1. **Toy codes only.** The primary benchmark is a one-dimensional repetition code; the second code is a surface-*like* toy lattice, not a stabilizer-exact CSS surface code, and we do not evaluate a full surface code at scale. The repetition code corrects only bit-flip errors and has a trivial (line) defect geometry, which is part of why the decoders cluster so tightly.

2. **Simulated structured noise.** All noise is produced by parametric samplers, not measured from hardware. The biased/burst/correlated/measurement-error regimes are plausible surrogates, but real device noise has correlations (leakage, crosstalk, drift) our samplers do not model.

3. **Logical-error differences are within noise.** The decoders are statistically indistinguishable on raw logical error at the scales we ran; the benchmark's value is in selectivity and calibration, not in resolving an accuracy ranking. Larger codes and more shots would be needed to separate decoders on accuracy.

4. **Single-distance focus.** The headline table is at one distance and one physical error rate; the threshold curves sweep p at a fixed distance but we do not present a distance-scaling (sub-threshold) study, which is the decisive test for a true topological code.

5. **Reduced-scope learned decoders.** The neural decoder is a logistic-regression coset selector and the RL decoder is a contextual bandit, not the deep graph or transformer decoders of the recent literature. Our calibration and selectivity findings are about these specific, lightweight models.

6. **Solo author, single environment.** All results were produced and audited by one author on a single CPU platform; independent replication on other hardware and by other groups remains to be done.

Conclusion. A deployable QEC decoder must do more than minimise an i.i.d. logical error rate: it must behave well under structured noise, know when to abstain, and report calibrated confidence. We built a reproducible benchmark that measures all three for a full decoder family

on one defect-graph interface, and we report—honestly—that the decoders are statistically indistinguishable on raw logical error while differing sharply in calibration and in the reliability they buy through abstention. The most actionable findings are that abstention cuts committed logical error several-fold for a moderate coverage cost, and that the best-calibrated decoder is the simplest one, not the most elaborate. Extending the framework to a full surface code at scale, to hardware syndromes, and to a distance-scaling study—while keeping selectivity and calibration as first-class metrics—is the natural next step.

3 Methods

3.1 Codes

The repetition code (`RepetitionCode`) has $n = d$ data bits and $d-1$ weight-two checks Z_iZ_{i+1}; the syndrome of an error $e \in \mathbb{F}_2^d$ is $s_i = e_i \oplus e_{i+1}$. The logical observable is the global parity $\sum_i e_i \bmod 2$. A correction $\hat{c}$ is counted as a logical failure unless the residual $r = e \oplus \hat{c}$ has zero syndrome (is a stabilizer) and zero logical value; a residual that does not even reproduce the syndrome is also a failure. The lookup decoder enumerates all 2^d errors to tabulate the minimum-weight leader per syndrome. The surface-like code (`SurfaceLikeCode`) places d^2 data bits on a grid with $(d-1)^2$ plaquette (weight-four) checks; defects are paired by Manhattan-distance matching and the logical observable is the parity of the left column. Both codes expose a bipartite Tanner graph via `networkx` [25].

3.2 Structured-noise samplers

Each sampler takes a seeded `numpy` generator [26] and returns a 0/1 error vector. *i.i.d.*: each bit flips at rate p. *biased*: even-index bits flip at $\min(1, \text{bias} \cdot p)$ and odd-index bits at p (default bias $= 3$). *burst*: scanning the line, with probability p a contiguous run of length $1 + \text{Poisson}(\text{burst_len} - 1)$ is flipped (default burst_len $= 3$). *correlated*: an i.i.d. seed pattern recruits each neighbour with probability corr (default 0.5) over two sweeps. *measurement_error*: i.i.d. data noise at rate p plus a per-check syndrome-flip mask at rate q (default 0.05), which the runner XORs into the measured syndrome.

3.3 Decoders

Every decoder implements `decode(syndrome)→(correction, confidence)`. *Lookup*: table from syndrome to minimum-weight leader; confidence boosted toward 1 since the leader is exact. *Majority*: cumulative XOR of the syndrome recovers a flip chain up to a global offset, choosing the lower-weight coset representative; confidence is the normalised majority margin. *Matching*: on the line, defects are paired left-to-right with boundary closure for an odd defect, returning the lighter coset; on the surface, a complete graph on lit plaquettes weighted by negative Manhattan distance is solved with `max_weight_matching` [9, 25] and an L-shaped path is flipped between matched pairs. Confidence is a defect-fraction heuristic $\exp(-2\, n_{\text{def}}/n_{\text{checks}})$, damped when a defect is unpaired. *BP*: a few rounds of min-sum message passing on the Tanner graph with log-likelihood priors; if the thresholded beliefs do not reproduce the syndrome, it defers to matching. *Neural*: a two-stage learned coset selector—a syndrome-determined base chain plus a `scikit-learn` logistic-regression classifier [27] predicting the logical coset of the true correction relative to that chain; the output is always syndrome-consistent and confidence is the predicted-class probability. *Topology*: matching with confidence recalibrated by defect-geometry features (count, span, gap structure). *Fused*: matching and neural both propose; agreement boosts confidence, disagreement defers to the more confident proposal with damped confidence. *RL*: a contextual bandit [28, 29] whose context is a coarse defect-count bucket and whose actions

are candidate decoders; it keeps per-context running success estimates, plays ε-greedy during a warm-up stream, and at evaluation selects the highest-value decoder per context.

3.4 Metrics and confidence intervals

The logical error rate is the mean of per-shot failure indicators; its 95% confidence interval is $1.96\,\hat{\sigma}/\sqrt{n}$ with $\hat{\sigma}$ the sample standard deviation (Bernoulli, so $\hat{\sigma} \approx \sqrt{\bar{p}(1-\bar{p})}$). The expected calibration error [23, 24] partitions shots into ten equal-width confidence bins and averages $|(\text{mean confidence}) - (\text{mean accuracy})|$ weighted by bin occupancy, where accuracy is $1-$failure. The risk–coverage curve sweeps an abstention threshold from the minimum to the maximum observed confidence; at each threshold, coverage is the committed fraction and risk is the logical error rate on committed shots. The curve is reported in descending-coverage order and a monotonicity check verifies that risk is (weakly) non-increasing as coverage shrinks.

3.5 Integrity: automorphism invariance and the audit gate

The repetition code is invariant under cyclic relabeling of its data bits (ring symmetry); a faithful decoder must yield the same logical error rate on cyclically rotated errors. The runner checks this for the matching decoder over shifts $\{1, 2, d/2\}$ within a statistical tolerance and exposes it as the `automorphism_invariant` flag. A separate audit module enforces that every reported number traces to a value in the generated `summary.json` (the single source of truth) and rejects forbidden absolute-superiority phrasing; this discipline is run in continuous integration.

3.6 Experimental protocol

A run builds the code, fits the neural decoder on a mixed-regime training stream, warms up the RL bandit on its own Monte-Carlo stream, then for each (regime, decoder) Monte-Carlos `trials` shots at the evaluation physical error rate p_{eval}, records the per-shot failure/confidence, and aggregates the metrics above. Threshold curves sweep p for the core decoders under i.i.d. noise; the risk–coverage curve is built from the fused and matching decoders' pooled confidences. Seeds, platform, library versions, runtime and peak memory are recorded as provenance in `summary.json`. All numbers in this paper are from the *reported-scale* configuration: distance 9, 40,000 trials per (regime, decoder) (200,000 pooled shots per decoder), 20,000 training shots, $p_{\text{eval}} = 0.12$, an eleven-point physical-error grid, run end-to-end in 507 s at 200 MB peak memory on a single CPU. A faster smoke configuration (distance 5, 1,500 trials, ≈ 13 s) is provided for continuous integration. Software: `numpy` [26], `scipy` [30], `networkx` [25], `scikit-learn` [27], `matplotlib` [31]; CPU only.

3.7 Per-regime results

The per-(regime, decoder) logical error rate, ECE and coverage at p_{eval} are written in full to `summary.json` under `by_regime`. At reported scale the qualitative pattern is regime-dominated: i.i.d. noise yields ≈ 0.002 logical error across decoders; biased ≈ 0.043; burst ≈ 0.17; correlated ≈ 0.21; measurement-error ≈ 0.34 (the hardest regime, since syndrome read-out flips corrupt the very signal the decoders condition on). Within each regime the decoders cluster within their confidence intervals, while coverage varies strongly by decoder: matching's coverage at the fixed operating threshold ranges from 0.28 (biased) to 0.67 (i.i.d.); topology and the RL selector also abstain substantially; majority, lookup, neural and fused commit fully (coverage 1.0) in every regime.

Data availability

This study uses no external datasets. All numerical values, tables, and figures are generated by the accompanying scripts from fixed random seeds and are written to a single source-of-truth artifact (`results/summary.json`); they are reproduced by running the pipeline.

Code availability

The code that implements every code, noise model, decoder, metric, and the audit gate, and that regenerates every figure, table, and reported number, accompanies this manuscript in `submission/code` and is intended for public release upon publication.

Competing interests

The author declares no competing interests.

Author contributions

M.H. conceived the study, designed and implemented the benchmark, decoders, noise models, metrics and audit pipeline, ran and analysed the experiments, and wrote the manuscript.

Funding

The author received no specific funding for this work.

Reproducibility statement

All experiments are reproducible on commodity hardware; runtime and memory are reported for each benchmark.

References

[1] Peter W. Shor. Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4):R2493–R2496, 1995.

[2] Michael A. Nielsen and Isaac L. Chuang. Quantum computation and quantum information. 2010.

[3] Barbara M. Terhal. Quantum error correction for quantum memories. *Reviews of Modern Physics*, 87(2):307–346, 2015.

[4] A. Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1): 2–30, 2003.

[5] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. Topological quantum memory. *Journal of Mathematical Physics*, 43(9):4452–4505, 2002.

[6] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3): 032324, 2012.

[7] Google Quantum AI. Suppressing quantum errors by scaling a surface code logical qubit. *Nature*, 614(7949):676–681, 2023.

[8] Oscar Higgott. PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. *ACM Transactions on Quantum Computing*, 3(3):1–16, 2022.

[9] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.

[10] David Poulin and Yeojin Chung. Iterative quantum coding systems and the topological color code. *Quantum Information & Computation*, 8(10):987–1000, 2008.

[11] Pavel Panteleev and Gleb Kalachev. Degenerate quantum LDPC codes with good finite length performance. *Quantum*, 5:585, 2021.

[12] Joschka Roffe, David R. White, Simon Burton, and Earl Campbell. Decoding across the quantum low-density parity-check code landscape. *Physical Review Research*, 2(4):043423, 2020.

[13] Giacomo Torlai and Roger G. Melko. Neural decoder for topological codes. *Physical Review Letters*, 119(3):030501, 2017.

[14] Savvas Varsamopoulos, Ben Criger, and Koen Bertels. Decoding small surface codes with feedforward neural networks. *Quantum Science and Technology*, 3(1):015004, 2017.

[15] Paul Baireuther, Thomas E. O'Brien, Brian Tarasinski, and Carlo W. J. Beenakker. Machine-learning-assisted correction of correlated qubit errors in a topological code. *Quantum*, 2:48, 2018.

[16] Stefan Krastanov and Liang Jiang. Deep neural network probabilistic decoder for stabilizer codes. *Scientific Reports*, 7(1):11003, 2017.

[17] Ryan Sweke, Markus S. Kesselring, Evert P. L. van Nieuwenburg, and Jens Eisert. Reinforcement learning decoders for fault-tolerant quantum computation. *Machine Learning: Science and Technology*, 2(2):025005, 2020.

[18] Philip Andreasson, Joel Johansson, Simon Liljestrand, and Mats Granath. Quantum error correction for the toric code using deep reinforcement learning. *Quantum*, 3:183, 2019.

[19] Johannes Bausch, Andrew W. Senior, Francisco J. H. Heras, et al. Learning high-accuracy error decoding for quantum processors. *Nature*, 635(8040):834–840, 2024.

[20] C. K. Chow. On optimum recognition error and reject tradeoff. volume 16, pages 41–46. IEEE, 1970.

[21] Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010.

[22] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. *Advances in Neural Information Processing Systems*, 30, 2017.

[23] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, pages 1321–1330. PMLR, 2017.

[24] Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015.

[25] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy)*, pages 11–15, 2008.

[26] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

[27] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[28] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, pages 661–670, 2010.

[29] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[30] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.

[31] John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

Supplementary Information

This Supplementary Information is a self-contained, from-first-principles derivation of every concept the accompanying code implements. It is written to be read top-to-bottom by someone who has seen linear algebra and basic probability but *not* quantum error correction, stabilizer codes, minimum-weight matching, belief propagation, calibration, or selective prediction. Every symbol used in the code is defined, and each section names the source file (under `submission/code/src/topoqec/`) it maps to, so theory and implementation can be read side by side. It expands on, rather than repeats, the Methods (Sec. 3); where a definition already appears there we cross-reference it.

S0.1 The problem in one paragraph

A quantum error-correcting (QEC) code stores one logical bit redundantly across many physical bits. Noise flips some physical bits; we are not allowed to look at the bits directly (that would destroy the quantum state), so instead we measure a few parity checks whose outcomes form a *syndrome*. A *decoder* takes the syndrome and guesses a correction. It succeeds if (correction $\oplus$ error) is a "do-nothing" operator (a stabilizer); it fails if that residual flips the logical bit. The usual figure of merit is the *logical error rate*. This project argues that two further axes matter just as much and are usually ignored: (i) *selective decoding*—letting the decoder *abstain* when unsure, so a slower fallback can take over; and (ii) *calibration*—whether the decoder's stated confidence actually matches how often it is right. We build one benchmark that measures all three (logical error, calibration, risk–coverage) for eight decoders across five structured-noise regimes, with honest 95% confidence intervals. The deliberately conservative headline: on this code the eight decoders are statistically *tied* on raw logical error, but they differ sharply in calibration and in the reliability that abstention buys.

S0.2 Codes, syndromes, and logical failure (`codes.py`)

Repetition code (`RepetitionCode`). The primary benchmark. A logical bit is encoded in d data bits over $\mathbb{F}_2 = \{0, 1\}$ (the field of integers mod 2; XOR is "+"). The code has $d - 1$ weight-two parity checks of the form $Z_i Z_{i+1}$. An error is a vector $e \in \mathbb{F}_2^d$ ($e_i = 1$ means data bit i is flipped). The syndrome of e is

$$s_i = e_i \oplus e_{i+1}, \qquad i = 0, \ldots, d-2 \qquad \texttt{(.syndrome())}. \tag{1}$$

Intuitively s_i fires exactly where an error *chain* crosses link i, so the lit syndrome bits sit at the *endpoints* of error chains. Decoding is therefore a matching/pairing problem on a one-dimensional line of links. The single logical observable is the global parity $L(e) = \left(\sum_i e_i\right) \bmod 2$ (`.logical_value()`). A correction $\hat{c}$ is a *logical failure* (`.logical_error()`) unless the residual $r = e \oplus \hat{c}$ both (a) reproduces zero syndrome (r is a stabilizer, acting trivially) and (b) has zero logical value; a correction that does not even reproduce the measured syndrome is automatically a failure. This is the precise definition used in Methods (Sec. 3.1) of "the code protected the bit".

Lookup table. For small d we enumerate all 2^d errors (`.all_errors()`) and, for every syndrome, store the minimum-weight error that produces it—the "coset leader". This optimal table is the gold-standard decoder for small d and the yardstick everything else is compared against (Sec. S0.5, `LookupDecoder`).

Surface-like code (`SurfaceLikeCode`). A second, two-dimensional code: $d \times d$ data bits on a grid with $(d-1)^2$ weight-four plaquette checks; a lit plaquette is a *defect*, and defects are paired by Manhattan (L_1) distance. The logical observable is the parity of the left column. It

exposes the same defect geometry in 2-D so the matching/topology decoders generalise, but it is a topology-*faithful* toy, *not* a stabilizer-exact CSS surface code—we say so honestly (Sec. 2.1). Both codes expose a bipartite Tanner graph via `networkx` (`.tanner_graph()`) so graph-based decoders operate on the code topology directly.

S0.3 Structured noise: why i.i.d. is not enough (`noise.py`)

Real hardware noise is *not* independent-and-identically-distributed. Five seeded samplers, each taking a `numpy Generator` and returning a 0/1 error vector, model the structure that matters (`noise.sample()` dispatches on the regime name; see Methods Sec. 3.2 for the parameters):

- `iid`: each bit flips independently with probability p (`sample_iid`).
- `biased`: even-index bits flip at $\min(1, \text{bias} \cdot p)$, odd-index at p (default bias = 3)—a coarse Pauli-bias surrogate (`sample_biased`).
- `burst`: scanning the line, with probability p flip a contiguous run of length 1+Poisson(burst_len−1) (default burst_len = 3)—spatially clustered errors (`sample_burst`).
- `correlated`: an i.i.d. seed pattern recruits each neighbour with probability corr (default 0.5) over two sweeps—neighbour correlations (`sample_correlated`).
- `measurement_error`: i.i.d. data noise at rate p *plus* a per-check syndrome read-out flip mask at rate q (default 0.05); the runner XORs that mask into the *measured* syndrome, so the decoder conditions on a corrupted signal (`sample_measurement_error`).

These are precisely the regimes where topology- or read-out-aware decoding might beat a memoryless majority vote—and the regimes turn out to dominate the difficulty far more than the choice of decoder (Sec. S0.9).

S0.4 What a decoder returns: the unified interface (`decoders.py`)

Every decoder implements one interface,

$$\texttt{decode(syndrome)} \longrightarrow (\text{correction, confidence}), \tag{2}$$

with correction in $\mathbb{F}_2^n$ and confidence in $[0, 1]$. The single interface is what makes the eight decoders comparable on one footing and what makes abstention (Sec. S0.7) and calibration (Sec. S0.8) definable uniformly. Two helpers recur: `_chain_from_defects()` turns a set of paired defect endpoints into a flip chain, and `_defect_confidence`$(n_{\text{def}}, n_{\text{checks}}) = \exp(-2\, n_{\text{def}}/n_{\text{checks}})$ is the defect-fraction confidence heuristic (more lit checks ⇒ less sure).

S0.5 The decoder family: textbook to learned (`decoders.py`)

- `LookupDecoder`: the optimal minimum-weight coset leader from the enumerated table; confidence boosted toward 1 since the leader is exact. The gold standard for small d.
- `MajorityDecoder`: cumulative XOR (prefix sum mod 2) of the syndrome recovers a flip chain up to a global offset; pick the lower-weight of the two cosets; confidence is the normalised majority margin.
- `MatchingDecoder`: minimum-weight perfect matching of defects. On the line, pair lit syndrome bits left-to-right (with a boundary closure for an odd count) and return the lighter coset. On the surface, build a complete graph on lit plaquettes weighted by negative Manhattan distance and call `networkx max_weight_matching` (Edmonds' blossom algorithm [9]),

then flip an L-shaped path between matched pairs. Confidence is the defect-fraction heuristic, damped when a defect is unpaired. This is the standard surface-code decoder, in toy form.

- `BPDecoder`: a few rounds of min-sum belief propagation (message passing) on the Tanner graph with log-likelihood priors; if the thresholded beliefs do not reproduce the syndrome, defer to matching. (BP underpins quantum LDPC decoding [10, 12].)
- `NeuralDecoder`: a learned two-stage coset selector. Stage 1: a syndrome-determined base chain (always syndrome-consistent). Stage 2: a `scikit-learn` `LogisticRegression` that predicts the logical coset (0/1) of the true correction relative to that base chain. The output is therefore always syndrome-valid and the confidence is the predicted-class probability—which is exactly why it tends to be well calibrated (Sec. S0.8). It is fit on a mixed-regime training stream.
- `TopologyDecoder`: matching, but with confidence *recalibrated* by defect-geometry features (count, span, gap structure).
- `FusedDecoder`: matching and neural both propose; agreement boosts confidence, disagreement defers to the more confident proposal with damped confidence (confidence fusion).
- `RLDecoder`: a contextual bandit (reinforcement learning). Context = a coarse defect-count bucket; actions = candidate decoders. It keeps per-context running success estimates, plays ε-greedy during a warm-up stream, and at evaluation picks the highest-value decoder per context.

`build_decoders()` wires the family together for a given code; the same list is summarised in Methods Sec. 3.3.

S0.6 Metric 1: logical error rate and honest intervals (`metrics.py`)

For each (regime, decoder) we Monte-Carlo many shots and record a per-shot failure indicator $F \in \{0, 1\}$. The logical error rate is $\hat{p} = \text{mean}(F)$ (`logical_error_rate()`). Because F is Bernoulli, its sample standard deviation is $s = \sqrt{\hat{p}(1-\hat{p})}$ and a 95% confidence-interval half-width is

$$\text{ci}_{95} = 1.96\,\frac{s}{\sqrt{n}} \qquad \texttt{(summarize())}, \tag{3}$$

with n the number of shots. This is the whole point of honesty: two decoders whose means differ by *less* than a CI width are *not* separated at 95%. At reported scale ($n = 200{,}000$ pooled shots) the eight decoders span only 0.1516–0.1540 with $\text{ci}_{95} \approx 0.0016$, so the 0.0024 best-to-worst spread is below one interval width: the decoders are statistically *tied* (Sec. 1.2). We report the tie, not a winner.

S0.7 Metric 2: selective decoding and the risk–coverage curve (`metrics.py`)

A decoder inside a fault-tolerant loop need not commit to every syndrome: it can *abstain* (flag a shot unreliable) so a fallback handles it. This is the QEC analogue of *selective prediction* in machine learning [20, 22]. Fix a confidence threshold τ and commit only on shots with confidence $\geq \tau$. Then

$$\text{coverage}(\tau) = \text{fraction of shots committed}, \tag{4}$$

$$\text{risk}(\tau) = \text{logical error rate on the committed shots}. \tag{5}$$

Sweeping τ from the smallest to the largest observed confidence traces the *risk–coverage curve* (`risk_coverage_curve()`), reported in descending-coverage order. As we abstain more (coverage shrinks), risk should fall: we drop the shots we were least sure about. `is_monotone_risk_coverage()` programmatically verifies risk is (weakly) non-increasing as coverage shrinks. Empirically the effect is large: matching's committed error falls from 0.1529 at full coverage to 0.0758 at ≈ 0.55 coverage and 0.0099 at ≈ 0.25 coverage (Sec. 1.3). The formal reason this works is Theorem 1: under perfectly calibrated confidence C (so $\Pr(\text{success} \mid C = c) = c$), the committed risk is $1 - \mathbb{E}[C \mid C \geq \tau]$, which can only decrease as τ rises—but *only* to the extent that confidence is calibrated. Hence Sec. S0.8.

S0.8 Metric 3: calibration (expected calibration error) (`metrics.py`)

A confidence score is only useful for abstention if it is *honest* about its own accuracy. The expected calibration error (`expected_calibration_error()`) partitions the shots into ten equal-width confidence bins; within each bin it compares the mean stated confidence to the mean realised accuracy (= 1−failure), takes the absolute gap, and averages those gaps weighted by bin occupancy:

$$\mathrm{ECE} = \sum_{\text{bins}} \frac{n_{\text{bin}}}{n} \left| \overline{\text{conf}}(\text{bin}) - \overline{\text{acc}}(\text{bin}) \right|. \tag{6}$$

ECE = 0 means "when the decoder says 0.8, it is right 80% of the time". The paper's most counterintuitive finding lives here: across the family ECE spans an eight-fold range (0.0317–0.2608) and is *uncorrelated* with sophistication (Sec. 1.4). The best calibrated is the lightweight `neural` selector (its confidence is literally a predicted-class probability, Sec. S0.5); the worst is plain `matching` (its defect-fraction heuristic tracks accuracy poorly). Calibration must be *measured*, not assumed from accuracy or complexity. This even contradicts the project's own README intuition that "fused" is best calibrated—we report the artifact's number, with fused ranking fourth at 0.0608, not the intuition.

S0.9 Structured noise separates regimes, not decoders (`runner.by_regime`)

The per-(regime, decoder) breakdown shows the dominant axis of variation is the *noise regime*, not the decoder (Sec. 1.5, Sec. 3.7). Logical error rises $\approx$ 100-fold from ≈ 0.002 (i.i.d.) to ≈ 0.043 (biased), ≈ 0.17 (burst), ≈ 0.21 (correlated) and ≈ 0.34 (measurement error, the hardest—readout flips corrupt the very signal decoders condition on). Within each regime the decoders again cluster inside their (now tight) CIs, while coverage varies strongly: matching/topology/RL abstain heavily under structure, whereas majority/lookup/neural/fused commit fully.

S0.10 Integrity: automorphism invariance and the audit gate (`symmetry.py`, `audit.py`)

Automorphism invariance. The repetition code is invariant under cyclic relabeling of its data bits (a ring symmetry). `cyclic_relabel(error, shift)` rotates an error; `logical_error_rate_under_relabeling()` and `is_automorphism_invariant()` check that the matching decoder yields the *same* logical error rate on rotated errors (over shifts $\{1, 2, d/2\}$, within a statistical tolerance). A faithful decoder must respect the code's symmetry; the run exposes this as the `automorphism_invariant` integrity flag (true at scale; Sec. 3.5).

Audit gate. `audit.py` enforces scientific hygiene: `audit_numbers()` checks every claimed number traces to a value in the generated `summary.json` (the single source of truth); `audit_reproducibility_phrase()` checks the mandated reproducibility statement is present; `audit_forbidden_claims()` rejects forbidden absolute-superiority phrasing. `scripts/audit_claims.py`

runs these in continuous integration and exits non-zero on any violation—so the paper cannot drift from the artifact.

S0.11 The single source of truth and how to reproduce (`runner.py`)

`runner.run()` ties it all together: build the code; fit the neural decoder on a mixed-regime training stream; warm up the RL bandit on its own Monte-Carlo stream; for each (regime, decoder) Monte-Carlo `trials` shots at p_{eval} and record per-shot failure/confidence; sweep p over a grid (i.i.d.) for the threshold curves; build the risk–coverage curve from the fused and matching confidences; check automorphism invariance. It writes `results/summary.json`—the single source of truth from which every table, figure and reported number is generated. Nothing is typed by hand. Then:

- `scripts/make_tables.py`: `summary.json` → `results/main_results.tex` (`booktabs`).
- `scripts/make_figures.py`: `summary.json` → the three figure PDFs `fig_schematic`, `fig_threshold` and `fig_risk_coverage`, in Nature Machine Intelligence house style (vector PDF, embedded editable text via `pdf.fonttype=42`, sans-serif with mathtext `dejavusans`, the colour-blind-safe Okabe–Ito palette, no in-plot titles, despined, with 95% CI bands).
- `scripts/audit_claims.py`: the readiness gate of Sec. S0.10.

To reproduce everything:

```
pip install -e .                      # installs topoqec from pyproject.toml
export KMP_DUPLICATE_LIB_OK=TRUE      # conda + pip OpenMP runtimes on macOS
bash scripts/reproduce_all.sh full    # reported scale (d=9, 40k trials/regime)
bash scripts/reproduce_all.sh         # fast smoke config for CI (d=5)
```

Reported run: repetition code, distance 9, five regimes, 40,000 trials/regime (200,000 pooled shots/decoder), 20,000 training shots, $p_{\text{eval}} = 0.12$, seed 0 (Methods, Sec. 3.6). The figure/table artifacts are copied verbatim into `submission/` to build the paper.

S0.12 What the result means (and what it does not)

The eight decoders are statistically *indistinguishable* on raw logical error for a distance-d repetition code in these regimes—as expected, since every reasonable decoder is close to the optimal coset leader and finite-sample noise dominates the residual gaps. We therefore make *no* accuracy-superiority claim. The genuine, separable contributions are: (i) the framework—a whole decoder family on one defect-graph interface under structured noise with honest CIs; (ii) selective decoding—abstention cuts committed logical error several-fold for a moderate coverage cost; and (iii) calibration as a first-class metric—ECE spans 8× and is decoupled from sophistication. Limitations are stated plainly in Sec. 2.1: toy codes only (repetition code; surface-*like* toy, not a stabilizer-exact surface code), simulated structured noise (not hardware syndromes), differences within noise on raw error, a single distance, lightweight learned models, and a single author/environment. Extending to a full surface code at scale, to hardware syndromes, and to a distance-scaling (sub-threshold) study—while keeping selectivity and calibration first-class—is the natural next step.